

看板 PDF 渲染验证 · 第 1 页

这一页是为了确认 pdf-page 块真的能把磁盘上的 PDF 画到卡片里而生成的。

它同时在验证三件事

要验证的	怎么看
文件路径能不能从宿主读到	这一页出现了，说明 /api/file 通了；出现「读不到」说明路径或权限有问题
中文字形正不正确	如果看到方块或乱码，说明 CMap 那条路没走通
页数对不对	看板上的块指定第 1 / 2 / 3 页，超出范围应当报「一共 N 页」

如果这一页显示成一条错误信息而不是纸张，那条信息本身就是要看的东西——它会同时出现在 board_outline 的 render failures 里。

第 2 页 • 表格与长文本

阶段三四个决定

问题	结论
UML 块的渲染归谁	自己带一份运行时（自带 mermaid，宿主路由下发）
PDF 与图片的入口	路径为主，一页一块，只加点名的页
页内锚点	现在不做，rect / point 留在模型里等 OCR
渲染错误怎么回到 Agent	客户端上报 + 宿主便宜校验，两条路汇进 board_outline

一段长文本，用来看换行与行距

看板本身不排版文档，它只是把一页纸原样显示出来。所以这一页的意义在于：如果它在卡片里看起来和在这里一样，那就说明栅格化的分辨率是对的，没有糊，也没有被拉变形。卡片会按自己的宽度重新栅格化，所以窗口拉宽之后它应该更清楚，而不是更模糊。

第二个用途是验证多页。同一个 PDF 的第 1 页和第 2 页是两个独立的块，但它们共用一次下载和一次解析——底下的文档缓存按路径去重。把第 2 页删掉再重新加上，不应该产生第二次网络请求。

第 3 页 · 边界

几个应该被看见的失败

- 指向一个不存在的路径 → 卡片说「读不到文件」并带上路径。
- 指向一个不是 PDF 的文件 → pdf.js 报结构错误。
- 页号写 99 → 卡片说「这份 PDF 一共 3 页，没有第 99 页」。

这三种都不会让看板崩掉，也都会走同一条上报通道，所以 Agent 下一轮就能在大纲里看到。

为什么不做页内锚定

要精确到「这一页左上角那个表」，先得有分块识别。需要专用解析模型（比如 deepseek-ocr-2 这类），现在不引入。模型里的 `rect` 和 `point` 锚点保留不动，将来接上就能用，不需要改模型。

—— 这一页是这一轮的最后一个验证点。